GraphRAG Explorer

Discovering how Austrian banks, owners and regulators are connected

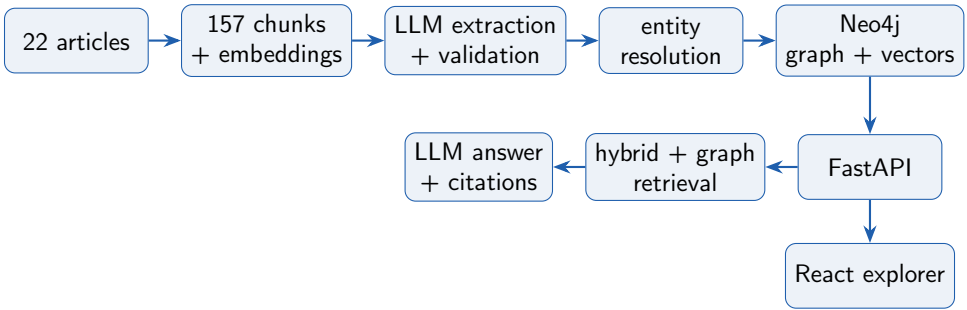
Kacper Zaleski

The goal

- A **runnable app** where people **visually explore** facts and connections they did not know
- From documents to a knowledge graph to an explorer UI, end to end
- Every fact **traceable to its source passage**
- Plus the honest question: does the graph make **answers** better than plain RAG?

Corpus	22 Wikipedia articles: Austrian banks, owners, regulators
Stack	Python, FastAPI, Neo4j, React + Cytoscape.js, Docker
LLMs	Gemini (free tier), switchable: Groq, Cerebras, Mistral

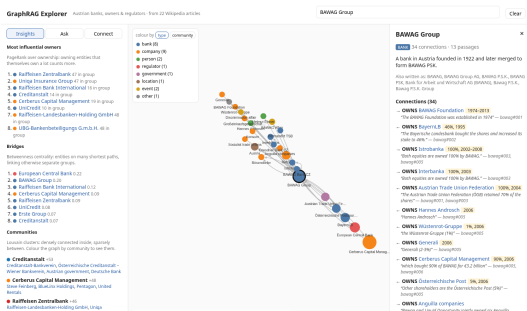
Architecture



- Ingestion is **idempotent and cached**: a re-run makes 0 API calls
- Result: **726 entities, 585 relations**, every edge knows its source chunks

Live demo

1. Insights: influential owners, communities
2. Entity card: BAWAG's ownership history
3. Connect: ÖGB ↔ GoldenTree
4. Connect: Refco ↔ Cerberus
5. Ask: *Who have been the owners of BAWAG over time?*
6. Ask: *How is BayernLB connected to Addiko Bank?*



Extraction: the LLM proposes, code verifies

- One LLM call per chunk, **Pydantic schema** as structured output
- Closed type lists: 8 entity types, 9 relation types
- One **canonical direction**: "A is a subsidiary of B" becomes `B OWNS A`
- Each item validated on its own: one bad relation does not drop the chunk
- **Grounding check**: the evidence quote must be supported by the chunk

What went wrong: exact-substring grounding rejected 73 relations, ~70 of them valid (quotes with "..."). Word-level support: 1 left, and it still catches the model dropping umlauts ("Österreichische" $\rightarrow$ "sterr ischische").

Entity resolution: false merges are worse than splits

- Normalise: case, accents, ö→oe, legal suffixes (AG, GmbH, a.s.)
- Then a **curated alias file**: every merge is explicit and reviewable
- Type by majority vote (the LLM says BANK once, COMPANY once)
- In a bank, merging two "Raiffeisen . . . " would **invent ownership**

What went wrong: the key chain ÖGB → BAWAG ← Cerberus was broken: the LLM wrote "Cerberus" and "ÖGB" in one chunk, full names in others. Also: "Ceberus" is a typo in Wikipedia itself.

Retrieval: graph mode

1. **Hybrid search**: vector + fulltext, fused with Reciprocal Rank Fusion (ranks, not scores)
2. **Seeds**: entities whose names are covered by the question
3. **Paths** between seeds, then breadth-first expansion with a node limit
4. Places are never stepping stones ("both are in Vienna")
5. Chunks behind the edges + reranker; **each path edge keeps a slot** for its evidence
6. Answer with citations; code removes citations of unseen passages

What went wrong: degree-based hub filtering blocked the main banks: here the high-degree nodes *are* the answer. Type (location) was the right signal.

Discovery features

- **Time and stake on edges**: a second, targeted LLM pass (15 calls, about 200 edges)
  BayernLB 46% (1995) → ÖGB 100% (2004) → Cerberus 90% (2006) → GoldenTree 21.8%
- **Graph analytics** at ingest (networkx):
  - PageRank over reversed ownership: who controls the most
  - Betweenness: bridges between groups (ECB, BAWAG)
  - Louvain communities: Erste + CEE banks, UniCredit + Bank Austria, ...
- Path finder without LLM: instant and free

What went wrong: "SIA" was a top-5 owner. "UniCredit sold its *card business* to SIA" had become `SIA ACQUIRED UniCredit`. One wrong edge distorted a global metric.

Engineering for unreliable free-tier APIs

- Timeouts, retries with backoff, client-side **rate limiter**
- **Fallback chain** of models on 503 and on quota errors
- **Content-addressed cache**: re-runs are free and reproducible
- All-or-nothing writes: a failed run writes nothing and resumes from the cache
- API: lifespan-managed resources, dependency injection, request ids, liveness vs readiness

What went wrong: `gemini-3.5-flash` allows **20 requests per day**; ingestion needs 160. The free tier also counts every text in an embedding batch as one request.

Evaluation design

- 28 questions: 13 multi-hop, 10 lookup, 5 corpus-wide
- Built from the **text, not the graph**: otherwise graph mode is graded on its own data
- Metrics: recall@8, complete@8, LLM judge, entity recall, latency, tokens
- Judge from **another vendor** (Groq) than the answers (Gemini): self-preference bias
- **Paired** statistics: bootstrap CIs, McNemar, Wilcoxon
- Judge reliability: second judge (Qwen), Cohen's κ

Round 1 (original prompt): no measurable difference

	vector	graph
judge score	0.91 [0.80, 0.98]	0.91 [0.84, 0.98]
correct	24/28	23/28
recall@8	0.90	0.89
LLM latency p50 / tokens	1.7 s / 2.8k	2.5 s / 3.4k

- Difference +0.00 [−0.05, +0.05], McNemar p = 1.0
- **Ceiling effect**: generated questions name their entities, vector finds both passages
- Both modes fail on **corpus-wide** questions: top-k cannot collect an aggregation

Round 2: a pre-registered hard test

- Hypothesis written **before** running: graph wins on connection questions
- 8 questions "How is A connected to C?", the bridge named nowhere
- Prompt fix evaluated only on these new questions: **no tuning on the test set**

	vector	graph
judge score	0.38	0.44
correct	3/8	2/8
recall@8	0.56	0.50
recall@8 **without reranker**	**0.81**	0.56

- Difference $+0.06$ $[-0.12, +0.25]$: **hypothesis not supported**
- The reranker hurts connection questions most (vector recall $0.81 \rightarrow 0.56$)
- Regression check on the old 28: nothing broke (graph 0.93 vs vector 0.91)
- Two judges agree on 51/56 + 13/16 answers, Cohen's $\kappa = 0.62$ / 0.69

What I learned

Where the graph helps

- Exploration: paths, neighbourhoods, rankings
- Explainability: every edge has a source quote
- Structure: time and stake on ownership
- Errors become visible (SIA, Cerberus)

Where it does not (yet)

- QA accuracy on a small, clean corpus
- Missing extractions cap it: "Addiko was spun off from Hypo" never became an edge
- Global questions need graph queries, not top-k

A negative result, measured honestly, is still a result.

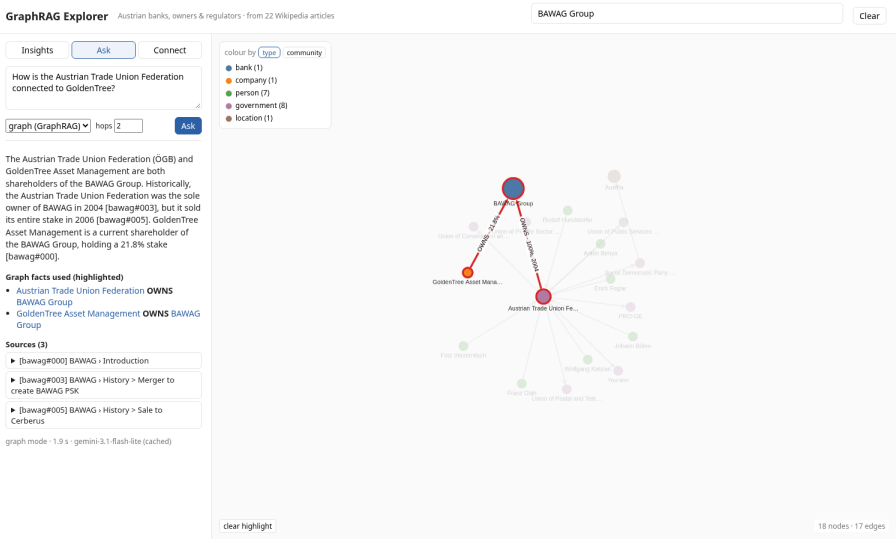
Next steps

- Drop or replace the reranker for graph mode (it is trained on web search queries)
- Aggregation mode for corpus-wide questions (graph query instead of top-k)
- Better extraction recall: a second pass on chunks with few relations
- Timeline filter in the explorer (the data is already there)
- In a bank: self-hosted or EU-hosted models, Snowflake as the document source, Airflow for scheduled re-ingestion

Thank you. Questions?

Code, evaluation and notes: `graphRAG-demo` repository

Backup: Ask with highlighted path



Backup: path finder

